

ideal for single-chip microcontrollers in embedded system design.

Graphical modelling using QM

By knowing the advantages of decomposing the software into event-driven actors, a developer might wonder how the quantum platform could be used to implement real-time embedded software. In order to help them, the package comes with a really cool graphical modelling tool, QP modeler (abbreviated as QM). The QM helps you elaborate graphically UML state charts associated with active objects. While using this tool, you need not re-invent the software architecture from scratch. The modeller helps you reuse the active object framework provided by QP. This will, in turn, increase your productivity dramatically.

QM also features automatic generation of C/C++ code, depending upon the framework used. Automatic model verification is also done during code generation. The tool provides mechanisms to quickly switch between the code and model, so as to apply required changes to the model than to the code. The QP/C and QP-nano frameworks comply with most of motor industry software reliability association's MISRA-C:2004 rules, while the QP/C++ framework complies with most of MISRA-C++2008 rules.

Feature highlights

Pre-emptive multitasking kernel. Users can go for the traditional RTOS for running the active-object model. But the QP framework provides a simple, superfast, non-blocking, run-to-completion (RTC) kernel. The kernel provides fully-pre-emptive multitasking using a single stack for all actor threads.

The pre-emptive kernel determines the scheduling policy where an RTC step can be pre-empted in order to execute another thread in the same processor. The suspended RTC step can later be resumed. There will be no concurrency issues as long as the pre-empting and pre-empted threads do not share the same resources.

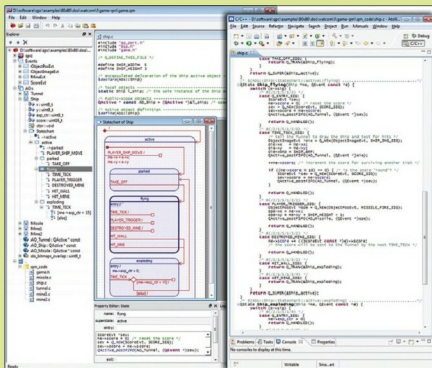


Fig. 3: Code generation in QM

Encapsulation for enhancing concurrency. Encapsulation involves packaging of data with operations into classes. It includes a mechanism for restricting access to some of the object's components as well as a language construct that facilitates bundling of data with the methods (or other functions) operating on that data.

Even without any mutual exclusion mechanism, all private attributes of the active objects can be accessed only by its own thread. In order to ensure proper encapsulation, the resources are not shared. A dedicated active object becomes the manager and access of the resources by the rest of the system is by the events posted to this manager-active object.

Highly-responsive event-driven architecture. There is an event queue associated with every active object. It helps them receive all the events exclusively through this queue. Whenever there are events like user actions, such as a mouse click, sensor outputs or messages from other threads, an event will be posted to the event queue asynchronously. The event producer need not wait in line for the actual processing of event. The QP framework takes up the responsibility of

Active objects (actors)

Actors are event-driven, encapsulated state machines running in their own thread of execution. They communicate with each other asynchronously by exchanging events. Behaviour is specified in QP by means of hierarchical state machines (UML state charts). They enforce the best practices of concurrent programming.

In response to a received message, an actor can:

- Determine its behaviour for the next message.
- Create a finite number of new actors.
- Send a finite number of messages to other actors.

safely delivering and queuing of events in the queue to the thread of recipient active object where the processing takes place. In the event-driven approach, the processor is busy only at the occurrence of the event. There is a better utilisation of resources and saving of power.

The event-driven approach seen in QP modeler is quite different from that seen in the traditional graphical user interface (GUI) development tools and architectures. Here, the actors not only make use of the nature of the event but also the context (state) of the system.